Levenshtein Distance

AlgoExpert · Medium · Dynamic Programming

The problem

- Minimum single-char inserts/deletes/substitutions turning a into b.

Intuition

- Dynamic programming over prefixes: each cell is the cheapest edit between two prefixes.

Brute force vs optimal

- Brute force: recursively try insert/delete/replace → O(3^(n+m))
- DP table over prefixes → O(n·m) time
- Keep only two rows → O(min(n, m)) space

Approach

- 1. Row zero and column zero are insert or delete costs
- 2. Equal characters copy the diagonal
- 3. Otherwise one plus the min of insert, delete, replace
- 4. Two rows suffice

Worked example

		y	a	b	d
	0	1	2	3	4
a	1				
b	2				
c	3				

Base row/column: edits from/to the empty string

Worked example

		y	a	b	d
	0	1	2	3	4
a	1	1	1		
b	2				
c	3				

'a' → 'ya': 'a' = 'a' → copy diagonal 1

Worked example

		y	a	b	d
	0	1	2	3	4
a	1	1	1	2	3
b	2	2	2	1	
c	3				

'ab' → 'yab': 'b' = 'b' → copy diagonal 1

Worked example

		y	a	b	d
	0	1	2	3	4
a	1	1	1	2	3
b	2	2	2	1	2
c	3	3	3	2	

'abc' → 'yab': 1 + min(neighbours) = 2

Worked example

		y	a	b	d
	0	1	2	3	4
a	1	1	1	2	3
b	2	2	2	1	2
c	3	3	3	2	2

'abc' → 'yabd': 1 + min(neighbours) = 2

Java solution

```java
public class LevenshteinDistance {
    public static int levenshteinDistance(String a, String b) {
        if (a.length() < b.length()) { String t = a; a = b; b = t; }
        int[] prev = new int[b.length() + 1], cur = new int[b.length() + 1];
        for (int j = 0; j <= b.length(); j++) prev[j] = j;
        for (int i = 1; i <= a.length(); i++) {
            cur[0] = i;
            for (int j = 1; j <= b.length(); j++)
                cur[j] = a.charAt(i - 1) == b.charAt(j - 1) ? prev[j - 1]
                        : 1 + Math.min(prev[j - 1], Math.min(prev[j], cur[j - 1]));
            int[] t = prev; prev = cur; cur = t;
        }
        return prev[b.length()];
    }
}
```

Complexity

- Time: O(n*m)
- Space: O(min(n, m))

Edge cases & pitfalls

- Empty strings: distance = other length
- Swap so the shorter string defines the row length
- Off-by-one: dp is (n+1) × (m+1)

Interview tips & follow-ups

- Explain the three neighbours: replace, delete, insert
- Follow-up: reconstruct the edits; weighted costs

Recap

- Dynamic programming over prefixes: each cell is the cheapest edit between two prefixes.
- Complexity: O(n*m)